API Access via Software Passkeys

Perry Kundert

2026-08-08

A WebAuthn assertion is a signature over `authenticatorData || SHA-256(clientDataJSON)`. Producing one needs a private key and about sixty lines of code – no browser, no human, no biometric. This memo specifies how a person whose Passkey lives in the macOS Passwords keystore provisions a *second* Passkey for a headless script, where every byte of that exchange flows, and what each side must remember afterwards.

The human's Passkey is a synced platform credential, reachable only through `navigator.credentials.get()`. It signs one kind of object and can never sign on the script's behalf; its role is to *authorize*, once, the creation of the API credential. That credential's private key is generated in the browser page, displayed once, and pasted into the script's `.env`. The server holds only a COSE public key – it never possesses a secret that authenticates. "Displayed once" describes the page, not the world: a value that has passed through a clipboard, a download or a backup has copies this design cannot account for, which is why browser minting is a *compatibility* path and generating the key on the target host is the safer shape.

At runtime the script performs the full WebAuthn ceremony in software, and *the same key* signs the per-request DPoP proofs (RFC 9449). That single-key choice yields the central result: the server stores **nothing** per session beyond the ordinary `sessions` row. The DPoP key thumbprint is *derived* from `credentials.public_key`, not stored, so the per-request check reduces to "this proof was signed by the same key that produced the assertion that opened this session." The new server state is small: a `jti` replay cache – structurally the same row the challenge table already is – plus a nonce-slot table when server-issued nonces are enabled, and a per-user registration generation used to fence revocation against in-flight registrations.

Contents

1 Status

Implemented on the `feature/machine-api-credentials` branch, and demonstrated end to end in `examples/demo`. Items still marked *proposed* below are the ones that were not built.

What exists:

- Schema version 2 — one step from the only released version — carrying `credentials.kind`, per-ceremony kind scoping on challenges, `credential_kind` on enrollment grants with the pending-grant index re-scoped by it, and the DPoP replay-cache and nonce-slot tables. Plus the versioned migration runner all three adapters needed before any of it could be added.
- `credentialKinds` configuration: `interactive`, `canRegister` and per-kind session lifetimes. Undeclared kinds — including `null` — behave exactly as before.
- `kind` reported on `SessionIdentity`, on both verification results, and on every credential and session event.
- `@localwebauthn/client`: the software authenticator, the two-line credential file, DPoP proof construction, and `MachineClient`.
- `verifyDpop` on the service, deriving the expected thumbprint from `credentials.public_key` so nothing is stored per session.
- `credential_kind` on enrollment grants, binding a bootstrap token to the class it may create, plus the pending-grant index scoped by it so provisioning a deployment key does not cancel a person's in-flight enrollment link.
- Credential heritage: `created_via`, `parent_credential_id`, `grant_id` and `approved_by_user_id`, with `credentialLineage`, `credentialDescendants` and `revokeCredentialTree`. The last is the remediation primitive for a stolen session that enrolled a passkey of its own — which `canRegister` does not prevent for a person, because adding a passkey while signed in is the feature. It crosses kinds unless given `kinds`, because an API credential's parent *is* the person's passkey: revoking a suspected passkey's tree stops their scripts too, which is right after a compromise and wrong the rest of the time.
- Kind filters on `revokeUserSessions` and `revokeUserAuthentication`, so signing a person out of their devices does not stop their nightly export.
- Optional server-issued DPoP nonces, keyed by time slot in the database so every server in a deployment converges on one value without a shared secret. The nonce is per *deployment*, not per credential or per session: its only job is to be unguessable in advance. Opt-in, because a hand-written client must retain the latest `DPoP-Nonce` and retry once when challenged.
- The demo's `/api/api-keys/*` and `/api/machine/v1/*` routes, an API-credentials UI panel, and `npm run api-demo` for the script side.
- `authenticateMachineRequest`: the one fail-closed call a machine route should use. It derives the session from the token, requires a proof, and refreshes the session's activity *only after* the proof holds — so a thief with the bearer token alone cannot keep a session alive. `verifyDpop` remains as the lower-level primitive.
- `dpopChallenge()` for the RFC 9449 section 8 header pair.

- A **registration fence**: a per-user generation stamped on every registration challenge and re-checked by the statement that commits the credential, so a revocation cancels ceremonies already in flight rather than racing them.
- Fixed-point revocation, which throws `revocation_not_converged` rather than reporting an incomplete remediation as success.

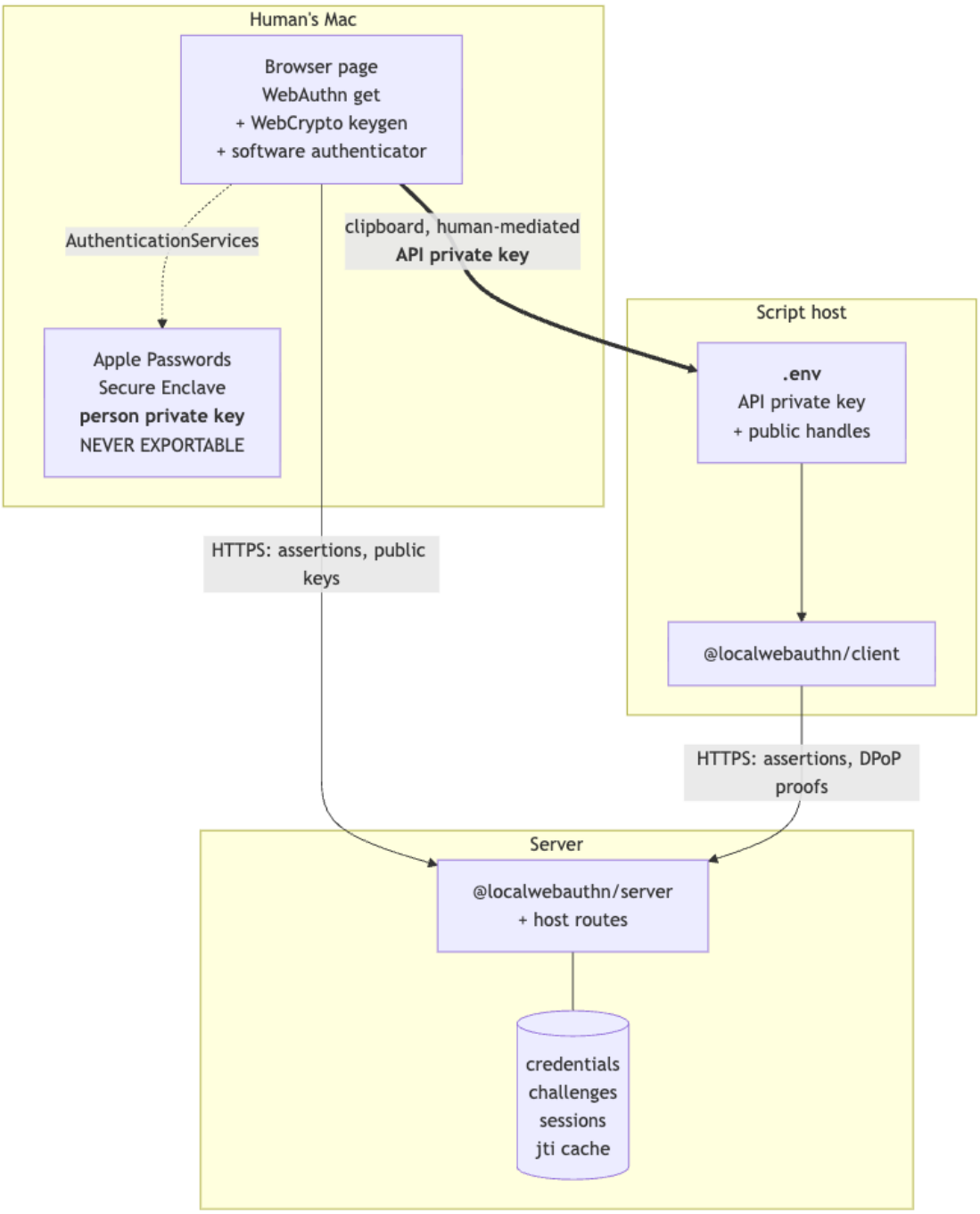
Not built, and still design only. Every *proposed* marker below refers to one of these:

- Keystore backends beyond a base64 PKCS#8 key. The `keystore:` URI is parsed and refused with a pointer to *Key Custody*.
- The `localwebauthn-client import` CLI that would move a key into a platform keystore.
- A client build-version header.
- The device-authorization (RFC 8628-shaped) provisioning flow, and target-host key generation generally. This document still describes browser minting as the primary flow because that is what ships; it is the *compatibility* path, and its hazards are in *Where the key material ends up*.

Companion documents in `docs/API-AUTH/`: three independent design treatments, a consensus analysis with the standards survey, and the provisioning discussion this document supersedes.

2 Overview

2.1 Principals



The thick edge is the only path the API private key ever travels: *page memory -> clipboard -> file*. It does not pass through the server, and the server has no column that could hold it.

2.2 The two credentials

Property	Human Passkey	API Passkey
Key custody	Apple Passwords, Secure Enclave	`.env` on the script host
Reachable from	a browser, only	any process that can read `.env`
`kind`	`person`	`service`
`label`	"Perry's MacBook"	"Perry's Blah maintenance script"
`deviceType`	`multiDevice`	`singleDevice`
`backedUp`	`true`	`false`
`signCount`	`0` forever; iCloud keeps no counter	`0` by policy
`userVerified`	`true`, attested by Touch ID	`true`, self-asserted by a program
`origin` in clientData	written by the *browser*	written by the *script*
Signs	assertions, browser-mediated	assertions + DPoP proofs

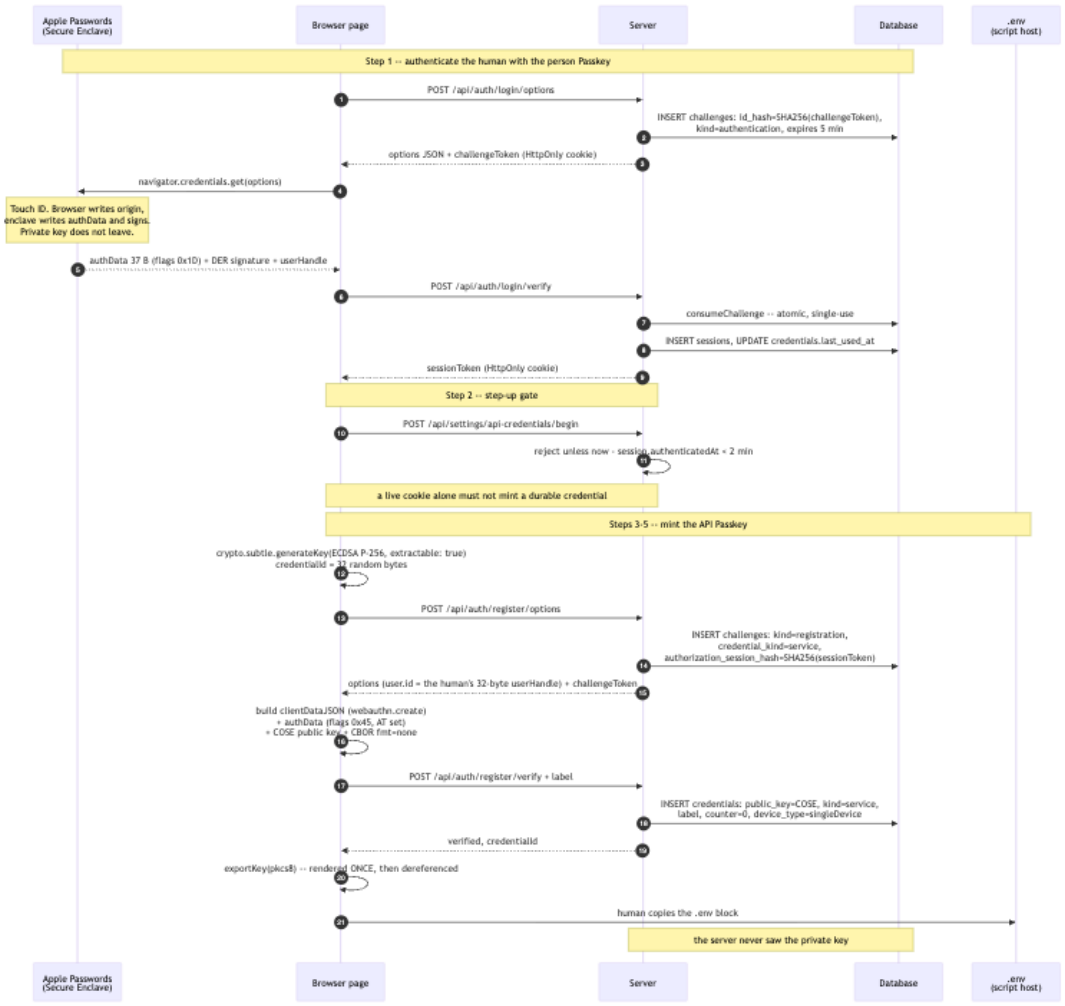
Both rows report `userVerified: true`, and it means something entirely different on each. That is what `kind` exists to record.

3 Part 1: A Human Provisions an API Passkey

3.1 Preconditions

- Server configured `rpId = app.example.com`, `expectedOrigins = ['https://app.example.com']`.
- The human holds a `kind`'person'= credential, private key in Apple Passwords.
- `getUser(userId)` returns them `active: true` with a stable 32-byte `webAuthnUserHandle`.

3.2 End-to-end data flow



3.3 Step 1: authenticate the human

Runs on	browser -> macOS -> Secure Enclave -> server
Code	`LocalWebAuthnBrowser.signIn()`, `auth.authenticationOptions()`, `auth.verifyAuthentication()`

The layer split *is* the phishing resistance, and it is why this key can never serve the script:

`BROWSER writes clientDataJSON:`

```
  {"type":"webauthn.get",
   "challenge":"<echoed verbatim from options>",
   "origin":"https://app.example.com",     <-- the BROWSER asserts this
   "crossOrigin":false}

AUTHENTICATOR writes authenticatorData, 37 bytes:
  [0..32)  SHA-256("app.example.com")
  [32]     0x1D = UP|UV|BE|BS
  [33..37) 0x00000000  signCount

AUTHENTICATOR signs, inside the enclave:
  ES256( authenticatorData || SHA-256(clientDataJSON) )   -- ASN.1 DER
```

Server verification, each step able to fail the call: consume the challenge atomically; look up the credential; require the user `active`; compare `response.userHandle` against `user.webAuthnUserHandle`; verify challenge, origin, `rpIdHash`, the UV bit and the signature; apply the `0 -> 0` counter rule; then commit the counter advance, the `last_used_at` touch and the session insert as one transaction.

3.4 Step 2: the step-up gate

```
const resolved = await auth.resolveSession(sessionToken);
if (!resolved) return unauthorized();

// SECURITY.md: a fresh assertion is required for sensitive credential changes.
// Minting a durable API credential is exactly that.
if (Date.now() - resolved.session.authenticatedAt > 2 * 60_000) {
  return json({ error: 'reauth_required' }, 401);   // browser repeats Step 1
}
```

`authenticatedAt` already exists on `SessionIdentity`, so this costs nothing. Without it, a stolen session cookie mints a credential that outlives the cookie you would revoke: the session becomes a credential factory.

3.5 Step 3: the page generates the API key pair

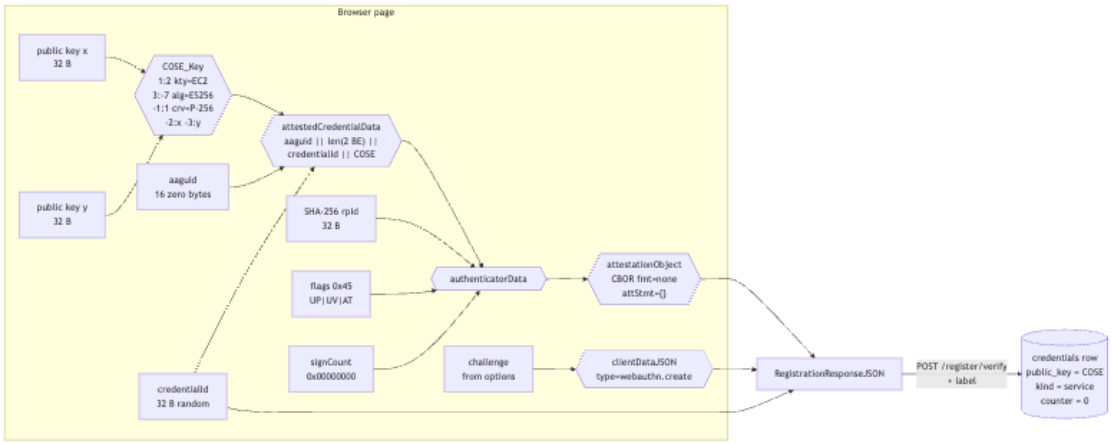
Calling `navigator.credentials.create()` here would mint a key *inside Apple Passwords* that can never be exported. What is wanted is an extractable software key, so the page implements a software authenticator itself:

```
const { publicKey, privateKey } = await crypto.subtle.generateKey(
  { name: 'ECDSA', namedCurve: 'P-256' },
  true,                                    // extractable -- the entire point
  ['sign'],
);
const credentialId = crypto.getRandomValues(new Uint8Array(32));
```

3.6 Step 4: assembling the registration response

The `credentialKind` is supplied by the *host route*, never read from the request body, and is written to the challenge row – so the credential's class is fixed on a server-owned row before the page ever

sees the challenge.



Two things worth noticing. `BE` and `BS` are deliberately clear, so the server records `singleDevice` / `backedUp: false` – honest for a key in a file. And with `fmt: "none"` there is **no attestation signature to compute at all**: the public key is simply asserted, and the first assertion is what proves the private half exists.

3.7 Step 5: one-time display, then transfer

Two data lines. The first carries every public value as compact JSON; the second is the key, or a reference to where the key really lives. A leading comment carries the label, which the client does not need.

```
# Perry's Blah maintenance script -- created 2026-08-08
LWA_CREDENTIAL='{"v":1,"baseUrl":"https://app.example.com","rpId":"app.example.com","origin":"ht
LWA_CREDENTIAL_KEY=MIGHAgEAMBMGByqGSM49AgEGCCqGSM49AwEHBG0wawIBAQQg...
```

3.7.1 `LWA_CREDENTIAL` – the public half

Field	Meaning
`v`	payload format version; `1` is this schema
`baseUrl`	origin the client talks to
`rpId`	hashed into `authenticatorData[0..32)`
`origin`	written verbatim into `clientDataJSON.origin`
`credentialId`	base64url, 32 bytes; the server's primary key for the row
`userHandle`	base64url, 32 bytes; required in `response.userHandle`
`alg`	COSE algorithm: `-7` ES256, `-8` EdDSA

Quoting rules matter here, and constrain the schema. The value is wrapped in single quotes so that a plain `source .env` does not let the shell strip the JSON's double quotes. Single-quoting in turn means no field value may contain an apostrophe – which is exactly why the human-authored `label` is *not* in the payload but in a comment. Every field above is a base64url string, a URL, a hostname, or a number, so the constraint holds by construction. Emit compact JSON with no spaces, and the line is safe for dotenv, `docker compose --env-file`, and shell sourcing alike.

Adding a field is a `v` bump. A client seeing a `v` it does not know must refuse rather than guess, because a future field could be load-bearing (an `audience`, say).

3.7.2 `LWA_CREDENTIAL_KEY` – the private half, or a pointer to it

One variable, two shapes, dispatched on the `keystore:` prefix:

```
# the key itself: base64 PKCS#8
LWA_CREDENTIAL_KEY=MIGHAgEAMBMGByqGSM49AgEGCCqGSM49AwEHBG0wawIBAQQg...

# or a reference -- .env then holds no secret at all
LWA_CREDENTIAL_KEY=keystore:keychain/localwebauthn/nightly-export
```

Base64 uses `+`, `/` and `=`, none of which need quoting, and a `keystore:` URI is likewise bare-safe. See *Key Custody* below for the backends.

The familiar "copy it now, you will not see it again" contract – with the improvement that, unlike a conventional API key, the server never had it to begin with. Mode `0600`, and in `.gitignore`.

3.8 Can the page write the file for you?

Worth asking, because "copy these two lines" is where a careful process usually goes wrong. Three mechanisms, and one of them does not exist.

3.8.1 Writing to a path the human picks

The File System Access API (`window.showSaveFilePicker()`) writes directly to a chosen location, so nothing transient is left behind. It is **Chromium only** – Chrome and Edge. Safari and Firefox do not implement it, and a person whose Passkey lives in Apple Passwords is quite likely to be in Safari. So it is an enhancement, never the only path.

3.8.2 Downloading a blob

`URL.createObjectURL` plus an `<a download>` works everywhere. Two caveats, one cosmetic and one not:

- A leading-dot filename is awkward: Finder hides it and Safari may append `.txt`. Offer `nightly-export.env` and let the human rename.
- The file lands in `~/Downloads`, which is readable by every process the human runs, Spotlight-indexed, and swept into Time Machine. A private key there is durable in a place people forget to look. `~/Downloads` is not iCloud-synced by default, which is the one mercy.

So a download is acceptable *with* a visible instruction to move the file and `chmod 0600` it – and it is strictly worse than pasting into an editor already open on the destination host, which creates no second copy at all.

3.8.3 Writing to the OS keystore – not possible from a page

There is no web API for the system keychain, and the near misses are all dead ends:

- `navigator.credentials` with `PasswordCredential` is Chromium-only, writes to the *browser's* password manager rather than the login keychain, is origin-scoped, and cannot be read by a separate process. Useless to a script.

- WebAuthn `create()` does put a key in the platform authenticator, but never returns the private half and will only ever produce assertions – which is the entire reason this design generates a software key instead.
- A custom URL scheme or native messaging host could bridge to the keychain, but that is an installed application, not a web page.

The division is therefore firm: **the page can deliver the material; only a process on the script host can place it in a keystore.** Hence a two-step, where the page's job ends at the file.

Proposed — no such command exists. Sketched here because it is the shape the `keystore:` URI anticipates, not because you can run it:

```
# PROPOSED, not implemented in v3.
localwebauthn-client import ./nightly-export.env \
    --keystore keychain/localwebauthn/nightly-export
# would read line 2, store the key in the keychain, and rewrite line 2 as
# keystore:keychain/localwebauthn/nightly-export
```

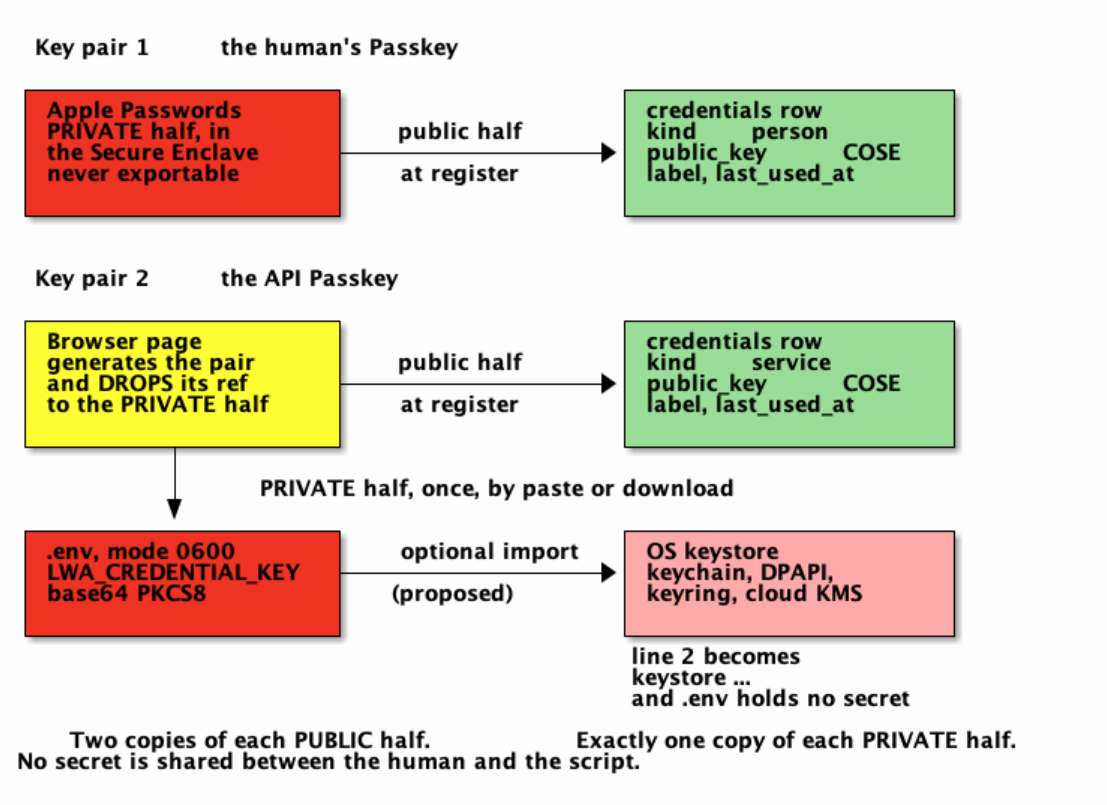
Note what such a command could *not* promise: erasing the original. Overwriting a file's bytes does not reach a copy-on-write snapshot, a journal, a backup, a clipboard history or an SSD's spare blocks, and no user-space tool can claim otherwise. Prefer never writing the key to a file at all — generate it on the target host, or have the keystore generate it — over writing one and hoping to unwrite it.

3.8.4 The constraint that decides which flow to use

Secure Enclave and TPM keys cannot be imported. They are generated in place, by `SecKeyCreateRandomKey` with `kSecAttrTokenIDSecureEnclave` or by the TPM itself, and no API accepts external key material into them. So the browser-generates flow in this document can reach `file`, `keychain`, `dpapi`, `keyring` and (for supported key specs) cloud KMS – but it can **never** reach the two strongest backends.

Those require the key to be born on the script host, which is the bootstrap-token flow in `docs/API-AUTH/history/PROVISIONING.md`. The two flows are complementary, not redundant: this one is simpler and works when the human is provisioning a script they cannot shell into; that one is the only route to a non-exportable key.

3.9 Where the key material ends up



4 Key Custody

`LWA_CREDENTIAL_KEY` holds either base64 PKCS#8 or a `keystore:` URI. The URI form keeps every secret out of `.env`, and for the hardware backends keeps it out of addressable memory entirely.

4.1 The URI scheme

`keystore:<backend>/<locator>[?<params>]`

URI	Exportable	Signs	Notes
`keystore:file//etc/lwa/nightly.key`	yes	ES256, Ed25519	Same trust as `.env`, b
`keystore:env/LWA_KEY_B64`	yes	ES256, Ed25519	Read from another va
`keystore:keychain/<service>/<account>`	yes	ES256, Ed25519	macOS login keychain
`keystore:dpapi//path/to/blob`	no[1]	ES256, Ed25519	Windows DPAPI, mac
`keystore:keyring/<collection>/<item>`	yes	ES256, Ed25519	Linux Secret Service c
`keystore:secure-enclave/<tag>`	**never**	ES256 only	P-256 only. Key must
`keystore:tpm/0x81000001`	**never**	ES256 only	TPM 2.0 persistent ha
`keystore:pkcs11/<module>?token=<label>&id=<hex>`	**never**	ES256 only	YubiHSM, SoftHSM, s
`keystore:awskms/<key-arn>`	**never**	ES256 only	`ECDSA_SHA_256`. Every
`keystore:gcpkms/<resource>`	**never**	ES256 only	`EC_SIGN_P256_SHA256`

4.2 The client interface

Everything above is one small interface, so the client never assumes the key is in memory:

```
type MachineKeyStore = {
  readonly algorithm: CoseAlgorithm;              // -7 (ES256) or -8 (EdDSA)
  publicKeyCose(): Promise<Uint8Array>;           // COSE_Key, for registration
  publicJwk(): Promise<JsonWebKey>;               // the DPoP proof's ‘jwk‘ header
  signWebAuthn(data: Uint8Array): Promise<Uint8Array>; // DER for -7, raw for -8
  signJws(data: Uint8Array): Promise<Uint8Array>;      // raw r||s for -7, raw for -8
};
```

Four methods rather than two, and the split is not cosmetic: the same key signs an assertion and a DPoP proof, and the two want **different signature encodings** — DER for WebAuthn, raw `r || s` for JWS. See *The signature-format trap.* A backend that exposed one `sign()` would force

[1] DPAPI protects the blob at rest with a machine or user secret; the plaintext key still passes through process memory when unwrapped, so it is "not exportable by an attacker without code execution", not "not exportable".

Three things fall out of that table and are worth stating plainly.

Every non-exportable backend is ES256-only. Secure Enclave, TPM, PKCS#11 tokens and both cloud KMSs do P-256 and not Ed25519. So choosing strong custody means choosing `alg: -7`, which means living with the DER-versus-raw conversion in *The signature-format trap.* Ed25519's nicer ergonomics are available only to software keys.

Non-exportable backends cannot be reached by this document's flow. See *The constraint that decides which flow to use* – they need the bootstrap-token flow.

A KMS backend changes the per-request calculus. Signing a DPoP proof per request against a network KMS adds latency, cost and quota to every API call. That is the one case where the single-key design should be revisited in favour of a separate, in-memory session key – see `docs/API-AUTH/history/CONSENSUS.md` section A. A Secure Enclave or TPM signature is local and fast enough that single-key remains correct.

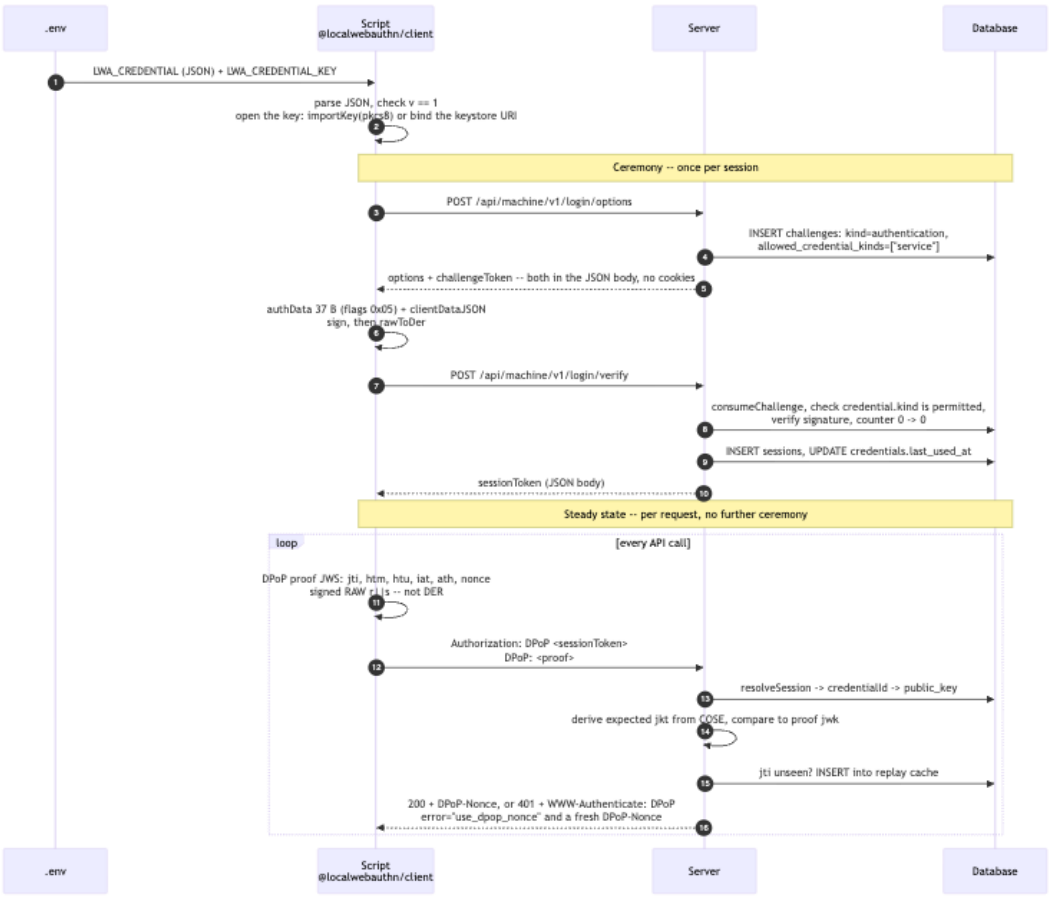
every caller to know which encoding the current operation wanted, which is exactly the mistake that costs an afternoon.

`publicJwk()` is separate for the same reason: the DPoP header carries a JWK, and deriving one from the COSE encoding is work every backend would otherwise repeat. Nothing here requires the key to be in memory — an agent, a TPM or a KMS satisfies this interface by signing bytes it is handed.

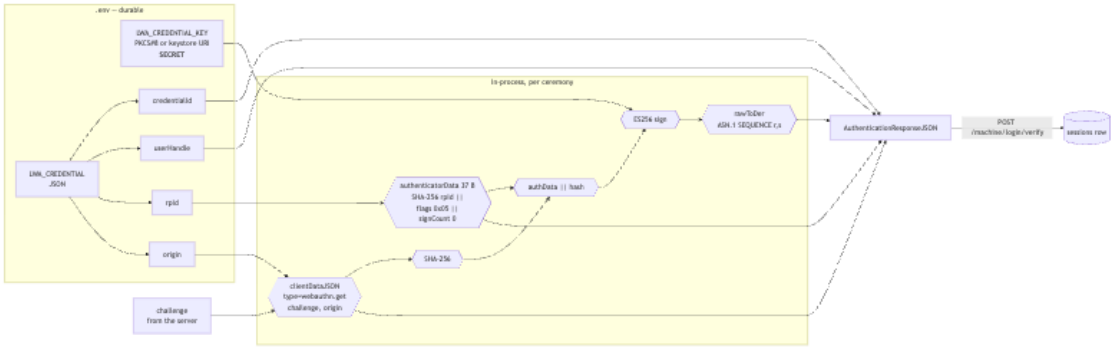
5 Part 2: The Script Opens an API Session

Each session is independent. The same `.env` credential can open any number of them concurrently.

5.1 End-to-end data flow



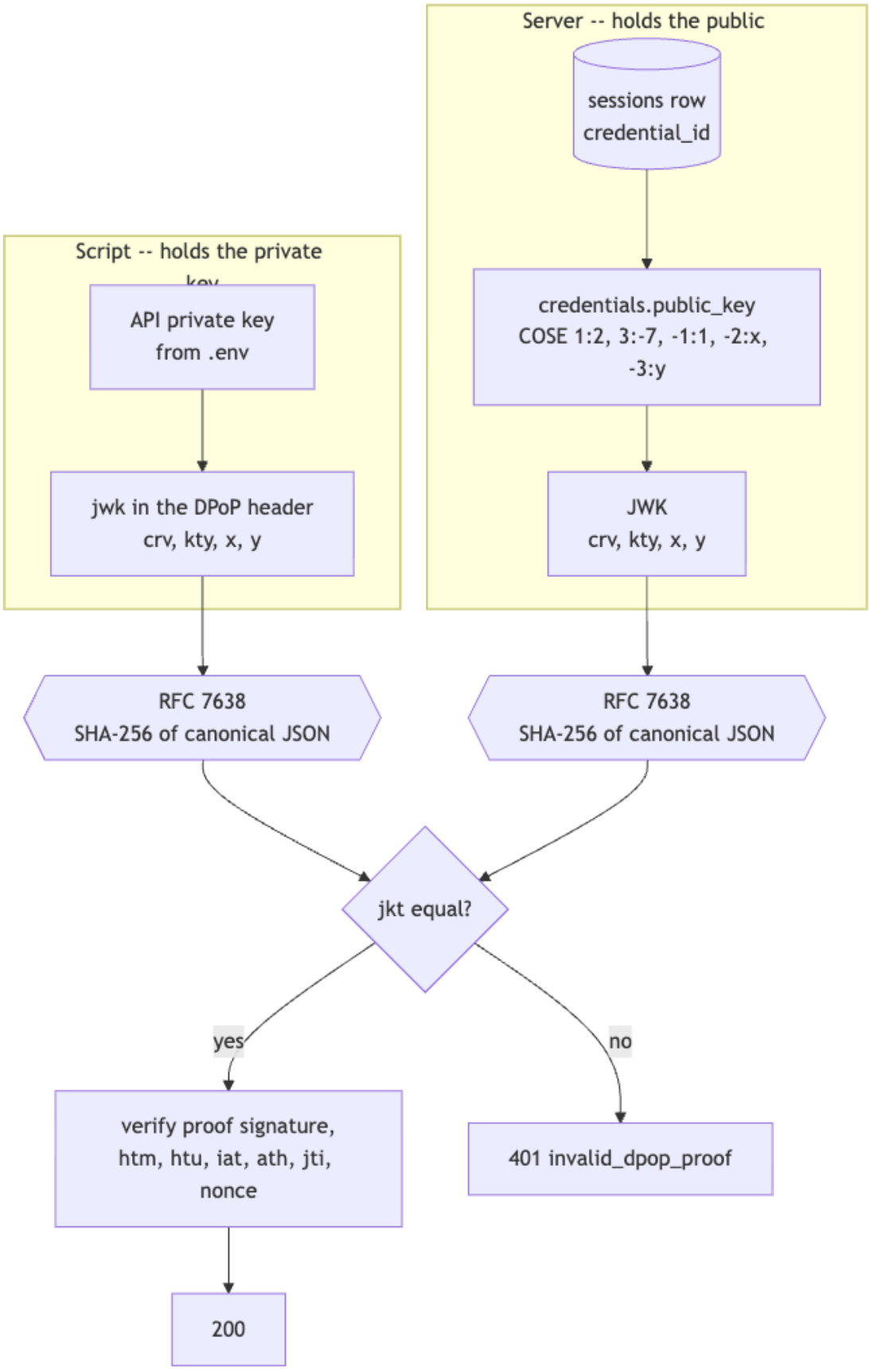
5.2 Assembling the assertion



5.3 Why the server needs no new per-session state

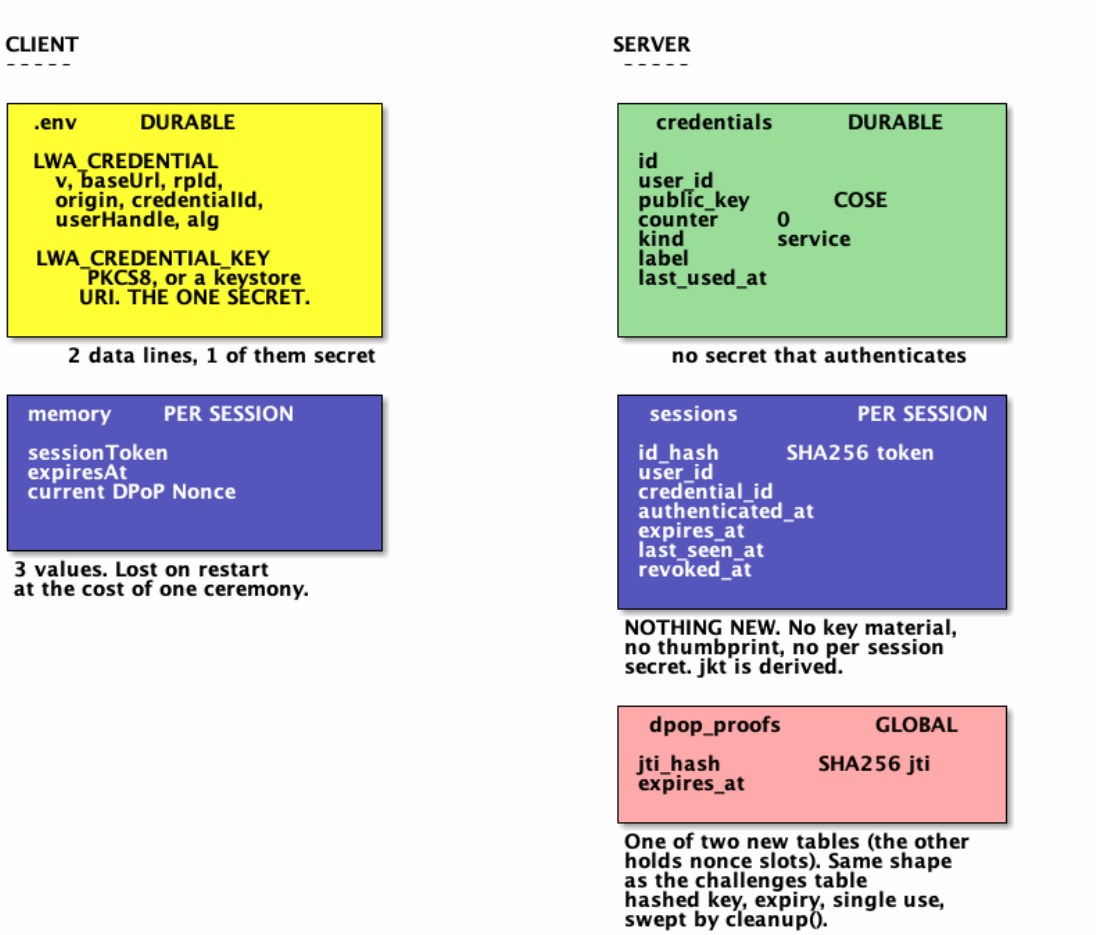
The API Passkey key and the DPoP key are the same key. The server already holds that key's public half in `credentials.public_key`, and the session row already records `credential_id`. So

the expected thumbprint is *derived*, never stored:


Server -- holds the public
sessions row
credential_id
credentials.public_key
COSE 1:2, 3:-7, -1:1, -2:x,
-3:y
JWK
crv, kty, x, y
Script -- holds the private
API private key
from .env
jwk in the DPoP header
crv, kty, x, y
RFC 7638
SHA-256 of canonical JSON
RFC 7638
SHA-256 of canonical JSON
jkt equal?
yes
no
verify proof signature,
htm, htu, iat, ath, jti,
nonce
401 invalid_dpop_proof
200

The check reduces to: *this proof was signed by the same key that produced the assertion that opened this session.* No `dpop_jkt` column, no issuance step, nothing generated per session. This is the direct answer to whether anything beyond the standard Passkey server-side confirmation is required: it is not.

5.4 What each side remembers



5.5 The DPoP proof

```
header  = {"typ":"dpop+jwt","alg":"ES256",
           "jwk":{"crv":"P-256","kty":"EC","x":"...","y":"..."}}
payload = {"jti":"<16 random bytes, base64url>",
           "htm":"POST",
           "htu":"https://app.example.com/api/reports",   -- no query, no fragment
           "iat":<unix seconds>,
           "ath":"<base64url(SHA-256(sessionToken))>",
           "nonce":"<last DPoP-Nonce seen>"}

signing input = base64url(header) || "." || base64url(payload)
signature     = ES256 over that -- RAW r||s, NOT DER
```

```
POST /api/reports HTTP/1.1
Host: app.example.com
Authorization: DPoP <sessionToken>
DPoP: eyJ0eXAiOiJkcG9wK2p3dCIs...
```

Server-issued nonces (RFC 9449 section 8) are worth enabling. They are the one control aimed squarely at a hostile-but-legitimate key holder: the client cannot pre-generate a proof for later use, because it cannot sign a nonce the server has not yet issued.

5.6 Concurrency and session lifetime

Unlimited simultaneous sessions, today, with no changes: `completeAuthentication` inserts a fresh row per ceremony and sessions share no mutable state.

Why `signCount` must be zero. With the counter pinned at `0` the compare-and-swap is `0 -> 0` and parallel ceremonies never contend. A strictly increasing counter would make concurrent sessions fight over one CAS, and the losers would get `409`. The requirement for many simultaneous sessions therefore *forces* `signCount = 0`. It also matches the human's Apple Passkey, which reports `0` forever, so any such deployment already tolerates it.

On binding a session to a TCP connection. Tempting, and not implementable. The client cannot see connection boundaries – Node's `fetch` pools and reuses connections through undici, and HTTP/2 multiplexes. The server cannot see them either: behind any reverse proxy it observes the *proxy's* connection. The one standard that truly binds to the connection, RFC 9729 Concealed HTTP Authentication, needs TLS-exporter access unavailable behind a terminating proxy, and emits an *identical* `Authorization` header for every request on that connection, so it is replayable within it.

What connection-scoping reaches for is a small blast radius for leaked session material. A per-request DPoP proof delivers that more completely: a captured token is useless without the key on **every** request, whatever the connection does. Pair it with a short `sessionAbsoluteMs` for the `service` kind and re-ceremony on `401`.

6 Restrictions on API Credentials

An API credential is a durable key in a file, held by a program, acting as a person. Everything it must *not* be able to do follows from one question: *could this capability be used to make the compromise of that file permanent, or to widen it beyond the script's job?*

The design principle constrains where each rule can live. A restriction is only real if the **server** enforces it – a client-side check is a suggestion. Of the rules below, the package must own the two that concern paths the package itself controls (registration, and which credentials a ceremony admits); the rest are host authorization decisions, which the package enables by reporting `credentialKind` on `SessionIdentity` and on every event.

6.1 The rules

#	An API session must not…	Enforced by	Mechanism
1	register any credential	**package**	`registrationOptions` / `verifyRegistrati`
2	authenticate at a browser endpoint	**package**	challenge-scoped `credentialKinds`; non-inte
3	revoke a credential other than its own	host	gate on `session.credentialKind`
4	revoke sessions other than its own	host	same
5	issue or exchange an enrollment grant	host	see below — the escalation this one prevents
6	satisfy a step-up gate for a human operation	host	step-up checks **kind and freshness**, not fre
7	receive a session cookie	host	machine routes never `Set-Cookie`
8	enumerate the user's other credentials	host	`listCredentials` is privileged
9	change account settings, channels, or recovery	host	account-takeover surface
10	count as the user's last credential	**package**	kind-scoped last-credential guard
11	present a bare Bearer token	host	resolve machine routes through `authenticat`
12	hold a long session	**package**	per-kind `sessionAbsoluteMs`
13	authenticate without being distinguishable	**package**	`credentialKind` on `credential.authentica`

6.2 Rule 5 is the one that actually bites

`canRegister: false` gates the **session** registration path. The **grant** path is a second route to registration, and the package cannot gate it at all: its authorization is possession of a single-use enrollment token, with no session involved, so there is no `credentialKind` to consult. Nothing in `registrationOptions({ enrollmentSessionToken })` can know whether a person or a program exchanged that token.

Which makes this chain the real hazard, and it is entirely host-side:

```
machine session token
  -> presented as a Cookie (a script writes Cookie and Origin freely)
  -> POST /api/clients/:id/enrollment
  -> enrollment token
  -> exchangeEnrollment -> registrationOptions({ enrollmentSessionToken })
  -> a brand new credential, canRegister:false fully bypassed
```

The demo had exactly this hole and it returned `200` with a live enrollment token. The fix has to be at the host's session middleware, because only the host knows who is calling `issueEnrollment`: `requireAuthentication` now refuses a session whose kind is declared `interactive: false`. Reusing that declaration rather than adding another switch keeps it to one rule — a kind that may not *open*

a session at the browser login route may not *use* one at a browser route either — and makes it fail-closed for every route that middleware guards, including ones added later.

Restrictions 3, 4, 8 and 9 are the same shape: all of them are host gates on a session, and all of them are satisfied by that one check. Rule 5 is singled out only because it is the one whose failure escalates rather than merely leaks.

6.3 The three that are easy to get wrong

Rule 1 is the self-replication hole. `registrationOptions({ sessionToken })` authorizes registration from *any* live session, which is the intended "add another passkey" feature for a human. The script's session is a live session. So without rule 1, a leaked `.env` key registers a spare credential and survives revocation of the first – revocation stops being a remedy at all. This is the one restriction that must be in the package, because `registrationOptions` is a package call and no host gate sits in front of it.

Rotation is the casualty. The tidy story of "authenticate with key A, register key B, revoke A" depends on exactly the path rule 1 closes. Rotation therefore goes back through a human-authorized grant, which is a real cost and the right trade.

Rule 6 is the subtle one. The step-up gate in Part 1 checks `now - session.authenticatedAt < 2 min`. An API credential can produce a fresh assertion at will, forever – so freshness alone is trivially satisfied and the API credential silently clears every step-up check in the application, including the one guarding creation of more API credentials. Step-up must assert *both*:

```
function requireFreshHuman(resolved) {
  if (resolved.session.credentialKind !== null) return false;   // rule 6
  return Date.now() - resolved.session.authenticatedAt < 2 * 60_000;
}
```

Rule 2 needs a fail-closed default, and the obvious one breaks hosts. Defaulting `authenticationOptions()` to "kind IS NULL only" is safe for today's deployments but locks out any host that backfills `kind`'person'= onto human credentials. Defaulting to "any kind" fails open, which is the whole thing we are trying to prevent.

The resolution is to declare it in configuration rather than at each call site, so the policy is visible in one place and a route cannot forget it:

```
new LocalWebAuthn({
  // ...
  credentialKinds: {
    service: {
      interactive: false,        // excluded from authenticationOptions() by default
      canRegister: false,        // rule 1
      sessionAbsoluteMs: 15 * 60_000,   // rule 12
    },
  },
});
// Rule 11 is enforced at the route, not by config: a machine route calls
// authenticateMachineRequest(), which always requires a DPoP proof.
```

Undeclared kinds – including `null` – behave exactly as today, so the change is monotone: an existing deployment that declares nothing sees no difference, and a host adopting service credentials declares them as part of adopting the feature. The machine route then opts in explicitly:

```
auth.authenticationOptions({ credentialKinds: ['service'] });
```

6.4 What is deliberately not restricted

- **Authenticating as often as it likes.** Concurrency is the point; see *Concurrency and session lifetime*.
- **Revoking itself.** A script that detects its own compromise should be able to kill its credential and its sessions. Rules 3 and 4 scope to *other* credentials.
- **Reading its own credential's metadata** – label, `createdAt`, `lastUsedAt`. Useful for a health check, and discloses nothing the holder does not have.
- **Business authorization.** What the script may do with the API is the host's, keyed on `credentialKind` and `credentialId`. The package takes no position, which is the `docs/RATIONALE.md` non-goal this design leaves intact.

6.5 API versioning

Four independent version numbers, which is three more than anyone wants but they move on different clocks:

Version	Owns	On a bump
`LWA_CREDENTIAL v`	the `.env` payload schema	client refuses an unknown `v` rather than guessi
`/api/machine/v1/`	the endpoint contract	old path served until its clients are retired
`LOCALWEBAUTHN_SCHEMA_VERSION`	the database schema	versioned migration, per *Open Items*
client build version (*proposed*)	diagnostics and EOL policy	not implemented; no version header is sent

Path-versioning the machine endpoints matters more than for browser endpoints: a browser reloads and gets the new client, whereas a script keeps its `.env` and its pinned dependency until somebody redeploys it, possibly for years. A credential minted against `v1` must keep working at `v2` or the versioning is decorative.

Have the client send its own version so operators can see what is actually deployed before they retire anything:

```
LWA-Client: @localwebauthn/client/1.4.2
```

That is diagnostics, not authentication – it is a self-asserted string like every other client claim, and per the design principle it must never gate a security decision. Refusing an EOL client on it is fine, because a client that lies about its version to keep working has only postponed its own breakage.

7 Implementation Notes

7.1 The signature-format trap

The same key signs in two incompatible encodings, and getting it wrong produces a verification failure with no diagnostic:

Context	Standard	ECDSA signature encoding
WebAuthn assertion	COSE alg `-7`	ASN.1 DER `SEQUENCE { r, s }`
DPoP proof (JWS)	RFC 7518 `ES256`	raw =r s=, 64 bytes

WebCrypto's `ECDSA` sign returns the raw form, so the WebAuthn path must convert to DER and the DPoP path must not. Ed25519 (`-8` / `EdDSA`) sidesteps this entirely – raw 64 bytes in both – and is worth preferring wherever the key store supports it.

7.2 Host routing

Machine routes must mount **outside** the starter's origin-check middleware. A script sends no `Origin` header, so `isExactOrigin(null, ...)` rejects it. That check exists to stop cross-site request forgery, which is a browser-only threat: CSRF needs ambient credentials a browser attaches automatically, and a script has none. Dropping it for machine routes loses nothing; the challenge and the DPoP proof carry the security there.

Cookies are likewise unused on machine routes – `challengeToken` and `sessionToken` travel in the JSON body and the `Authorization` header. No service change is needed for this: `authenticationOptions()` already *returns* `challengeToken` as a value, and only the browser routes choose to wedge it into a cookie.

7.3 What this design does not need

- No OAuth authorization server, no authorization codes, no token endpoint.
- No JWT *issuance*. The only JWT is the client-made DPoP proof, verified with the key it embeds – so no server signing key, no JWKS endpoint, no key rotation.
- No `dpop_jkt` column; derived from `credentials.public_key`.
- No per-session server secret.
- No attestation verification: `fmt: "none"`, and nothing a signer asserts about itself is evidence anyway.

8 Open Items

1. **Machine sessions must not enroll credentials.** `registrationOptions({ sessionToken })` authorizes registration from any live session. The script's session is a live session, so a leaked `.env` key could register a spare credential and survive revocation of the first – revocation stops being a remedy. Default to refusing session-path registration when the authorizing session's credential `kind` is non-null; rotation then goes back through a human-authorized grant.

2. **Kind-scoped last-credential guard.** Without it the script's credential counts as "another active credential", letting you revoke the human's only Passkey while reporting success.

3. **Versioned migrations.** `LOCALWEBAUTHN_SCHEMA_VERSION` is `1` and the migration runner is `CREATE TABLE IF NOT EXISTS`, which cannot add a column. Prerequisite for every schema change here.

4. **Per-kind session durations**, so `service` sessions can be minutes while human sessions stay hours.

5. **Kind-scoped pending-grant index** – only if the bootstrap-token variant is also supported. The flow above uses the session path and issues no grant.

9 Cross-References

Everything under `docs/API-AUTH/history/` predates the implementation and none of it was updated afterwards: where one contradicts this document, this document is right. They are kept for the reasoning, not the conclusions.

- `docs/API-AUTH/history/DESIGN-MACHINE-CREDENTIALS.md` – the `kind` column, the byte formats, threat model against API keys, fleet topology.
- `docs/API-AUTH/history/CONSENSUS.md` – reconciles three designs; surveys RFC 9449 (DPoP), RFC 9421 (HTTP Message Signatures), RFC 9729 (Concealed), and `draft-ietf-oauth-first-party-apps`; argues for the mechanism over the OAuth machinery.
- `docs/API-AUTH/history/PROVISIONING.md` – the bootstrap-token variant, where the script rather than the browser generates the key pair.
- Issue #11 – the `kind` column this design depends on.